


# SISTEMAS OPERATIVOS

08011442

**Germán Sánchez Torres, I.S., M.Sc., Ph.D.**

Profesor, Facultad de Ingeniería · Programa de Ingeniería de Sistemas

Universidad del Magdalena, Santa Marta

Teléfono: +57 605 421 4079 ext. 1138 · 301 683 6593

Edificio Docente, cubículo 3D401

Correo: gsanchez@unimagdalena.edu.co · sanchez.gt@gmail.com




# Jerarquía de memoria

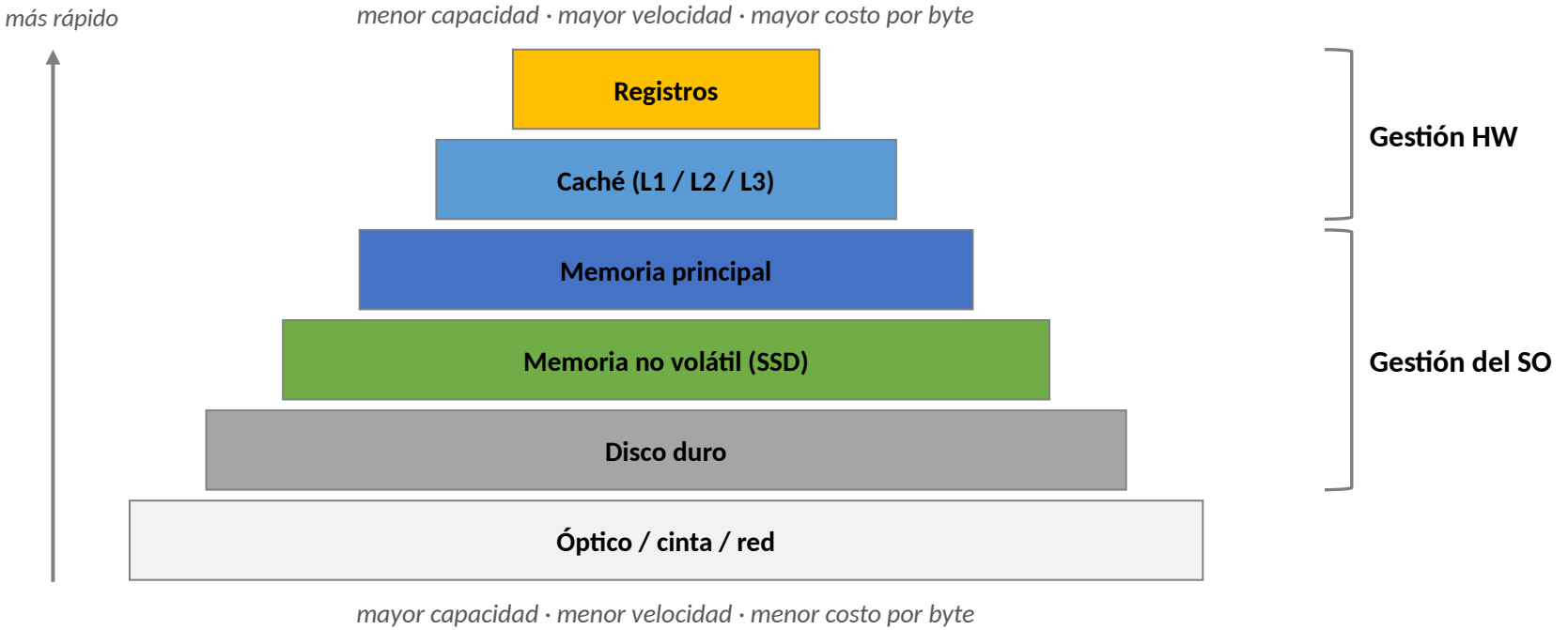
# Contenido

1. Introducción y objetivo
2. Niveles de la jerarquía
3. Migración de la información
4. Políticas del mecanismo de migración
5. Direccionamiento entre niveles
6. División y traducción de la dirección
7. Proximidad referencial
8. Proximidad espacial, secuencial y temporal

# Introducción y objetivo

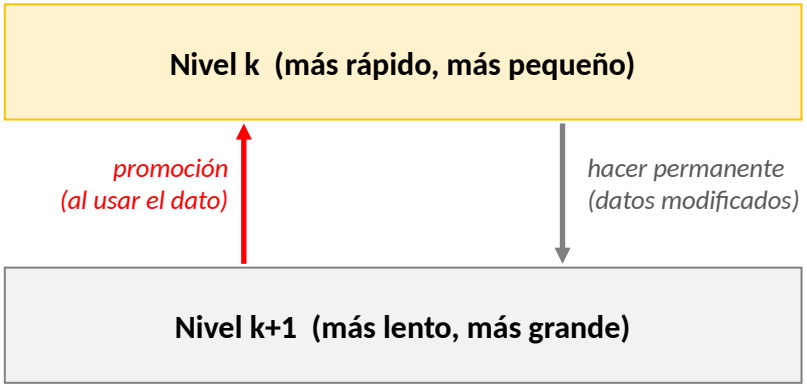
- **Objetivo:** conseguir tiempos de acceso aceptables a costos razonables.
- Ningún dispositivo de memoria es a la vez grande, rápido y barato: la jerarquía combina varios niveles para aproximarse a las tres cosas.
- Los elementos de información se **replican** en varios niveles. Cuando un dato se va a usar, se promueve hacia los niveles más rápidos (hacia arriba); cuando se modifica o crea en un nivel rápido y debe hacerse permanente, se lleva a los niveles inferiores.
- **Problema de coherencia:** hay que actualizar las distintas copias de un mismo dato, de forma que se garantice que siempre se use el valor correcto.
- **Problema de traducción:** las copias del mismo dato, en niveles distintos, tienen direcciones diferentes; hay que traducir entre ellas.

# Niveles de la jerarquía de memoria



# Migración de la información

- **Objetivo:** que la información esté en el nivel adecuado cuando se necesite.
- La migración se puede realizar de dos formas:
  - **Automática:** transparente al programa. Ocurre en la memoria caché y en la memoria virtual.
  - **Por demanda explícita:** el propio programa solicita el movimiento de la información.




*El nivel k está más cerca del procesador; el k+1 tiene mayor capacidad, pero es más lento.*

**Automática:** una caché sube un bloque al acceder a él.
**Explícita:** un programa hace `read()` de disco a memoria.

# Políticas del mecanismo de migración

- El mecanismo de migración se basa en tres aspectos:

| Tamaño de bloque | Política de extracción | Política de reemplazo |
|---|---|---|
| Cantidad de información movida entre niveles. Disminuye a medida que se sube en la jerarquía (hacia la cúspide). | Qué información sube al nivel k desde el nivel k+1, y cuándo. La solución más común es por demanda: se promueve lo que el programa referencia y, aprovechando la proximidad espacial, también las direcciones cercanas (bloque, línea, página). | El nivel k es más pequeño que el k+1, así que estará lleno cuando se quiera llevar información a él: hay que decidir qué datos presentes sacar. |

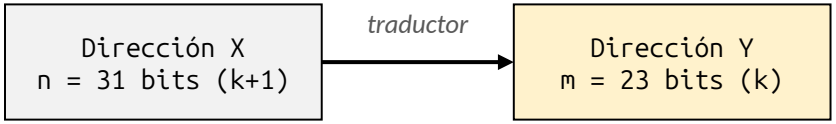
Ejemplo: en una caché, el bloque es una línea de caché; la extracción es "traer al fallar"; el reemplazo sigue políticas como LRU (el menos usado recientemente).

# Direccionamiento entre niveles

- El programa en ejecución genera una dirección X relativa al dato *A*, que se refiere al nivel k+1.
- Si hay una copia del dato en el nivel k, esa copia tiene otra dirección, Y: hay que traducir de X a Y.
- El problema no es trivial: puede haber notables diferencias de tamaño entre niveles.

**¿Cuántos bits necesita una dirección en cada nivel?**

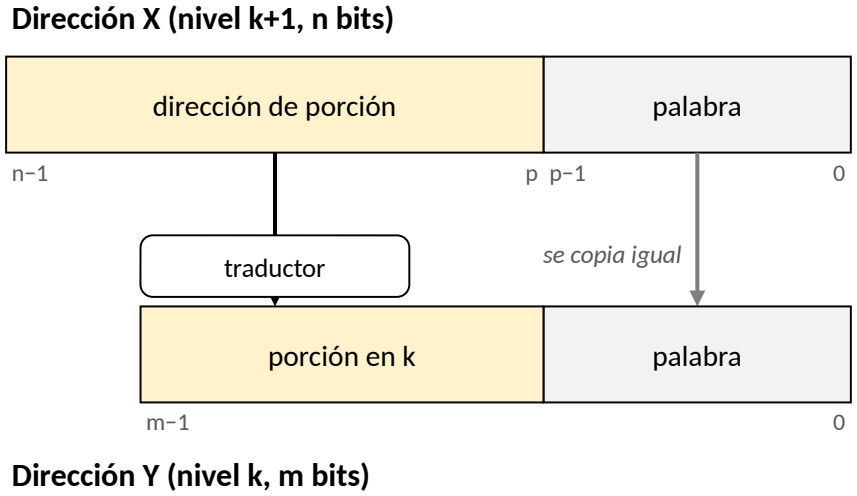
| Nivel | Tamaño | Bits de dirección |
|---|---|---|
| k + 1 | 2 GiB = $2^{31}$ bytes | `n = 31` |
| k | 8 MiB = $2^{23}$ bytes | `m = 23` |




**Fórmulas:** $2^n = x \Leftrightarrow n = \log_2 x$. Cambio de base: $\log_b x = \log_k x / \log_k b$.

# División y traducción de la dirección

- Para simplificar la traducción y aprovechar la proximidad espacial, los mapas de direcciones se dividen en porciones de tamaño fijo ($2^p$ bytes).
- Estas porciones (bloques) son la unidad mínima de información transferida entre niveles.
- Cada dirección se descompone en dos partes:
  - **Bits altos**: identifican la **porción** ($n - p$ bits en X; $m - p$ en Y).
  - **Los *p* bits bajos**: identifican la **palabra** (el byte) dentro de la porción.




La eficacia de la jerarquía se mide con la **tasa de aciertos** (hit rate) y el **tiempo medio efectivo** de acceso.

# Proximidad referencial

- Un programa en ejecución solo usa, en cada momento, una pequeña parte de toda la información que maneja.
- Las referencias usadas muestran ciertas relaciones entre sí: no son aleatorias.
- Se llama **traza** a la lista ordenada, en el tiempo, de las direcciones de memoria referenciadas. La traza de un programa incluye tanto direcciones de instrucciones como de datos.

```
Re = re(1), re(2), re(3), ..., re(j)

re(i):  i-ésima dirección referenciada por el programa e
```

- Dadas dos direcciones *u* y *v*, se define su distancia como `d(u,v) = |u – v|`.
- Así, la distancia entre dos referencias de la traza es `d(re(j), re(k)) = |re(j) – re(k)|`.

# Proximidad espacial, secuencial y temporal

- **Proximidad espacial:** dadas dos referencias re(j) y re(i) próximas en el tiempo (*i* – *j* pequeño), existe alta probabilidad de que su distancia `d(re(i), re(j))` sea también pequeña.
  - **Proximidad secuencial:** muchos fragmentos de programas y estructuras de datos se recorren de forma secuencial: es probable que la referencia siguiente a re(j) sea la siguiente dirección de memoria.
- **Proximidad temporal:** los programas suelen referenciar direcciones usadas en el pasado reciente. Es probable que la próxima referencia, re(j+1), esté entre las *n* anteriores:

```
re(j-n+1), re(j-n+2), ..., re(j-1), re(j)
```

Estas formas de proximidad son las que justifican que la memoria caché funcione: guardar los bloques "cercanos" a lo último accedido reduce mucho las fallas.

# Resumen

- La jerarquía combina niveles rápidos y pequeños con niveles grandes y lentos, para acercarse a "rápido, grande y barato".
- La información migra entre niveles de forma automática (caché, memoria virtual) o por demanda explícita del programa.
- El mecanismo de migración se define por el tamaño del bloque y las políticas de extracción y de reemplazo.
- Cada dirección se divide en bloque (porción) y palabra; el traductor convierte direcciones entre niveles.
- La jerarquía funciona porque los programas exhiben proximidad espacial, secuencial y temporal: no acceden a la memoria al azar.

# Referencias y lecturas


- Carretero, J. et al. *Sistemas operativos: una visión aplicada*, cap. 1: jerarquía de memoria. McGraw-Hill, 2007.
- Silberschatz, A., Galvin, P. y Gagne, G. *Operating System Concepts*, 10.ª ed., cap. 1. Wiley, 2018.
- Stallings, W. *Computer Organization and Architecture*, 10.ª ed., cap. 4: memoria caché. Pearson, 2016.
- Patterson, D. y Hennessy, J. *Computer Organization and Design*, 5.ª ed., cap. 5: la jerarquía de memoria. Morgan Kaufmann, 2014.